

Relatório — Trabalho Prático de Sistemas Operacionais

Pseudo-SO Multiprogramado

Universidade de Brasília — Profa. Aletéia Patrícia Favacho de Araújo

Lucas Licio (211043342) • Eduardo Volpi (190134330) • Moises Altounian (200069306)

Este trabalho consistiu na implementação de um pseudo-Sistema Operacional multiprogramado, composto por um Gerenciador de Processos, um Gerenciador de Memória, um Gerenciador de E/S e um Gerenciador de Arquivos. O simulador recebe três arquivos de entrada (processos, operações de arquivo e strings de referência de páginas) e reproduz o comportamento do sistema: o escalonamento da CPU, a paginação com memória virtual, a alocação exclusiva de recursos e as operações sobre o sistema de arquivos.

1. Ferramentas e linguagens utilizadas

A implementação foi feita integralmente em **Python 3** (testada com a versão 3.10), utilizando **apenas a biblioteca padrão** da linguagem — especificamente os módulos `sys` (leitura dos argumentos de linha de comando) e `collections.deque` (filas de prontos com inserção e remoção em tempo constante). Nenhuma biblioteca de terceiros foi empregada, em conformidade com a exigência da especificação de que apenas os padrões da linguagem sejam utilizados. O código é compatível com ambiente UNIX/Linux e é executado pelo interpretador por linha de comando:

```
python3 dispatcher.py processes.txt files.txt string.txt
```

2. Descrição teórica e prática da solução

O programa foi dividido em módulos coesos, cada um responsável por uma abstração do sistema, conforme sugerido na especificação. O processo *despachante* (`dispatcher.py`) é o processo principal: ele lê as entradas, cria os processos, cede a CPU a cada um e imprime todas as informações de saída.

2.1. Gerenciador de Processos e Escalonamento

Cada processo é representado por um bloco de controle (classe `Processo`), que guarda seus dados estáticos e o estado dinâmico. O escalonamento adota duas classes distintas. Os processos de **tempo real** (prioridade 0) são geridos por uma fila **FIFO sem preempção**: uma vez em posse da CPU, executam até o término. Os processos de **usuário** utilizam **múltiplas filas de prioridade com realimentação** (MLFQ), com três níveis (prioridades 1 a 3) e **quantum** de tempo. Ao gastar seu quantum, o processo é rebaixado para a fila imediatamente inferior (realimentação); para evitar *starvation*, aplica-se **envelhecimento** (*aging*): processos que esperam além de um limite são promovidos a uma fila de maior prioridade. Segue-se a convenção de que menor valor numérico indica maior prioridade. As filas são implementadas em `filas.py` e a política de decisão no laço do escalonador, em `dispatcher.py`.

2.2. Gerenciador de Memória

A memória principal é simulada por **paginação** com 20 *frames* de 1 KB. Dos 20 frames, 8 são reservados exclusivamente aos processos de tempo real e 12 aos de usuário; as duas áreas nunca se misturam, o que garante o isolamento entre as classes de processo. Como cada processo possui sua própria lista de frames, um processo nunca acessa a região de memória de outro. Durante a execução, cada página referenciada que não está residente gera uma **falta de página**, contabilizada por processo. A substituição de páginas usa o algoritmo **LRU** (*Least Recently Used*) em escopo **local**, com pré-carga de uma página. Toda essa lógica está em `memoria.py`.

2.3. Gerenciador de E/S

O sistema administra a alocação e a liberação de quatro tipos de recurso: 1 scanner, 2 impressoras, 1 modem e 2 dispositivos SATA. Cada recurso é alocado a um processo por vez (uso exclusivo) e não há

preempção na alocação de dispositivos. A sincronização é feita por **semáforos** contadores (operações *wait/signal*), representados na classe `Semaforo`. A alocação é atômica — o processo adquire todos os recursos solicitados de uma só vez ou nenhum, evitando retenções parciais. Processos de tempo real não requisitam recursos de E/S. O módulo correspondente é `recursos.py`.

2.4. Gerenciador de Arquivos

O sistema de arquivos representa o disco como um vetor de blocos e utiliza **alocação contígua**: cada arquivo ocupa blocos consecutivos. A busca por espaço livre emprega o algoritmo **first-fit**, sempre a partir do primeiro bloco. As permissões seguem a especificação: processos de tempo real podem criar (havendo espaço) e deletar qualquer arquivo, enquanto processos de usuário só podem deletar arquivos que eles mesmos criaram. Arquivos já presentes no disco no início da simulação não possuem criador e, portanto, só podem ser removidos por processos de tempo real. Ao final, imprime-se o mapa de ocupação do disco (blocos vazios são indicados por 0). O módulo é `arquivos.py`.

3. Principais dificuldades encontradas e soluções

3.1. Divergência na contagem de faltas de página do processo P0

O exemplo do enunciado indica 6 faltas de página para o processo P0. Nossa implementação, aplicando LRU local, obteve **7** faltas. Ao investigar, verificamos que, para a string de referência do P0 (1,2,3,4,1,2,5,1,2,3,4,5) com 4 frames, o LRU produz 8 faltas em demanda pura, ou 7 quando se desconsidera a pré-carga da primeira página. O valor 6 só é alcançado pelo algoritmo **ótimo** (que substitui a página cujo próximo uso é mais distante), e não pelo LRU. Como a especificação determina explicitamente o uso do LRU, concluímos que **7 é o resultado correto** e que o número apresentado no exemplo provavelmente foi obtido por outro critério. Essa conclusão é reforçada pelo processo P1, para o qual nossa implementação produz exatamente 14 faltas — valor idêntico ao do enunciado —, evidenciando que a lógica do LRU está correta. **Solução:** mantivemos o LRU conforme a especificação, deixando explícita e isolada no código a convenção de pré-carga (a primeira página não é contabilizada como falta), e registramos a divergência.

3.2. Processo sem recursos disponíveis: reenfileirar em vez de rebaixar

Ao implementar o escalonamento dos processos de usuário, surgiu uma dúvida conceitual sobre como tratar um processo que é escolhido para executar mas não pode iniciar por não haver todos os recursos de E/S disponíveis no momento. A hipótese inicial foi rebaixá-lo de fila, como ocorre quando um processo gasta seu quantum. Após análise, percebeu-se que isso seria incorreto: o processo **não chegou a executar**, logo não há razão para penalizá-lo na prioridade. **Solução:** o processo é apenas recolocado no fim da fila de mesma prioridade (mantendo sua prioridade) e o relógio avança, de modo que ele seja reavaliado quando os recursos forem liberados. O rebaixamento (realimentação) ficou reservado exclusivamente aos processos que efetivamente consumiram um quantum de CPU.

3.3. Inconsistência no exemplo do sistema de arquivos

No arquivo de operações do exemplo, a operação 5 é “1, 0, E, 2”, cujo código 0 indica **criação**. Contudo, a saída esperada apresentada no enunciado descreve uma tentativa de **deleção** que falha. Trata-se de uma inconsistência do próprio exemplo. **Solução:** testamos as duas possibilidades — mantendo o código 0 (criar), e havendo espaço livre nos blocos 8 e 9, o programa cria o arquivo E com sucesso; apenas ao trocar para o código 1 (deletar) a saída passa a coincidir com a do PDF. Seguimos a regra geral (o código da operação determina a ação) e tratamos a divergência como um erro de digitação do enunciado.

3.4. Definição do dono de arquivos pré-existent

A especificação não define quem é o proprietário dos arquivos que já se encontram gravados no disco no início da simulação. Como a regra de permissão estabelece que um processo de usuário só pode deletar arquivos criados por ele mesmo, essa omissão precisava ser resolvida para decidir se tais arquivos

poderiam ser removidos por processos comuns. **Solução:** adotamos a interpretação de que os arquivos pré-existent não pertencem a nenhum processo (dono = None). Como consequência, apenas processos de tempo real — que, por regra, podem deletar qualquer arquivo — conseguem removê-los; processos de usuário comum ficam impedidos, pois não constam como criadores. A decisão mantém o controle de permissões consistente com o restante da especificação.

4. Papel de cada integrante

- **Lucas Licio (211043342):** módulos de Processos (bloco de controle), leitura das entradas, Memória (paginação e LRU), Filas e Escalonador (tempo real FIFO e MLFQ com envelhecimento) e o Despachante (integração e impressão da saída); testes de integração e validação contra o exemplo do enunciado.
- **Eduardo Volpi (190134330):** Gerenciador de Arquivos — alocação contígua, busca first-fit, operações de criação e deleção, controle de permissões e impressão do mapa de ocupação do disco.
- **Moises Altounian (200069306):** Gerenciador de E/S — modelagem dos recursos (scanner, impressoras, modem e SATA), alocação exclusiva sem preempção e sincronização por semáforos.

5. Uso de Inteligência Artificial

O uso de ferramentas de IA no desenvolvimento foi o seguinte:

- **Claude (Anthropic):** geração de casos de teste (*testbenches*), verificação de indentação e padronização de comentários, e formatação deste relatório.
- **GitHub Copilot:** sugestões de autocompletar durante a codificação, com a lógica revisada e validada manualmente pelos autores.

6. Bibliografia

- TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016.
- STALLINGS, W. *Operating Systems: Internals and Design Principles*. Pearson Education, 2009.
- MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.